Visiting a 2-D table in memory

Memory map

Consider a tiny version of the table, with 3 drivers and 4 hours per week. Each cell gives the offset in bytes of a value from the beginning of the table.

	`[0]`	`[1]`	`[2]`	`[3]`
`waits[0]`	0	4	8	12
`waits[1]`	16	20	24	28
`waits[2]`	32	36	40	44

Tiny cache

Consider a tiny processor cache, which works as follows:

- A cache line is 16 bytes, i.e., exactly one row of the tiny table.
- The cache has a capacity of a single line.
- Reading an `int` whose line is in the cache is free. Otherwise, its line must be fetched from memory, and replaces the previous one.

Visiting order

Without running anything, write down the offset of each value, in the order in which each function visits them, and whether its line must be fetched (yes or no).

- `TotalsHourByHour()`: column by column, i.e., hour by hour.
- `TotalsDriverByDriver()`: row by row, i.e., driver by driver.

Visit	**`TotalsHourByHour()` offset**	**Fetch?**	**`TotalsDriverByDriver()` offset**	**Fetch?**
1	0	yes	0	yes
2	16	yes	4	no
3				
4				

Visit	`TotalsHourByHour()` offset	Fetch?	`TotalsDriverByDriver()` offset	Fetch?
5				
6				
7				
8				
9				
10				
11				
12				